

Tokenization, Cost Optimization & Semantic Embeddings

Hands-On Lab Guide

tiktoken · HuggingFace Tokenizers · OpenAI Embeddings · SentenceTransformers

AWS EC2 Ubuntu · Jupyter Notebook · Code Server

Module	Topic	Tools
Module 1	Visualize Tokenization of Insurance Documents	tiktoken, HuggingFace Tokenizers
Module 2	Token Cost Calculation & Prompt Optimization	tiktoken, OpenAI Pricing API
Module 3	Embeddings & Semantic Similarity for Policy Clauses	OpenAI Embeddings, SentenceTransformers

Use Case Scenario

- You are an AI Engineer at InsureTech Solutions building an LLM-powered policy analysis system.
- The system must process thousands of insurance policy documents — auto, health, and life policies.
- Today's lab covers three foundational skills: understanding how tokens work across LLM models,
- calculating and reducing prompt cost, and computing semantic similarity between policy clauses.

Prerequisites

- AWS EC2 Ubuntu instance (t3.medium or better) with Jupyter Notebook and/or Code Server running
- Python 3.9+ installed · pip package manager available
- OpenAI API key set in environment: `export OPENAI_API_KEY='sk-...'`
- Internet access from the EC2 instance to download packages and call APIs

■ Environment Setup

Before running the labs, install all required Python libraries in your EC2 terminal or a Jupyter cell.

Step 1 **Open a terminal on your EC2 instance (SSH or Code Server terminal)**

Step 2 **Install all dependencies in one command**

```
pip install tiktoken transformers sentence-transformers openai \
matplotlib seaborn pandas numpy scikit-learn \
torch --index-url https://download.pytorch.org/whl/cpu
```

Step 3 **Set your OpenAI API Key as an environment variable**

```
# In terminal (add to ~/.bashrc for persistence)
export OPENAI_API_KEY='sk-your-actual-key-here'

# Verify it is set
echo $OPENAI_API_KEY
```

Step 4 **Launch Jupyter Notebook and create a new notebook**

```
# If Jupyter is not already running
jupyter notebook --ip=0.0.0.0 --port=8888 --no-browser &

# Open browser and navigate to: http://<EC2_PUBLIC_IP>:8888

# Create a new notebook: insurance_ai_lab.ipynb
```

■■■ Ensure port 8888 is open in your EC2 Security Group inbound rules. For Code Server users, the terminal inside VS Code works identically.

■ Module 1 — Visualize Tokenization of Insurance Documents

What You Will Learn

- Understand what tokenization is and why it matters for LLM APIs (you pay per token, not per word).
- Use tiktoken (OpenAI's tokenizer) to tokenize insurance documents for GPT-3.5 and GPT-4 models.
- Use HuggingFace Tokenizers to tokenize the same documents for BERT and GPT-2.
- Visualize token boundaries with color-coded HTML output inside Jupyter.
- Compare token counts across four different tokenizers on the same insurance text.

Concept: What Is a Token?

A token is the basic unit of text that LLMs process. Tokens are not words — they are sub-word fragments determined by a tokenizer's vocabulary. The word 'deductible' might tokenize into ['ded', 'uct', 'ible'] (3 tokens). Short common words like 'the', 'is' are usually 1 token. API pricing is based on tokens (input + output), so understanding tokenization directly impacts cost.

Word Count	GPT-4 Tokens	Approx. Ratio
5 words	~8 tokens	1.6x
4 words	~9 tokens	2.25x
100 words	~130-140 tokens	1.3x
100 words	~150-170 tokens	1.5-1.7x

Step-by-Step Implementation

Step 1

Create sample insurance documents — paste this into Cell 1 of your notebook

```
# Cell 1 — Sample Insurance Documents

insurance_docs = {
    "auto_policy": """
    AUTOMOBILE INSURANCE POLICY — Policy No. AUT-2024-78432
    Coverage Type: Comprehensive and Collision
    Insured Vehicle: 2022 Toyota Camry (VIN: 4T1BF1FK0CU123456)
    Annual Premium: $1,842.00 | Deductible: $500
    Coverage Period: January 1, 2024 to December 31, 2024
    """
}
```

```

Liability Limit: $100,000 per occurrence / $300,000 aggregate
Exclusions: Racing, commercial use, intentional damage, war perils.
Claims must be reported within 30 days of the incident.
""",
"health_policy": ""
HEALTH INSURANCE POLICY – Group Plan ID: HLT-GRP-2024-001
Plan Type: PPO (Preferred Provider Organization)
Monthly Premium: $425.00 | Annual Deductible: $1,500 individual
Out-of-Pocket Maximum: $5,000 per calendar year
Pre-existing conditions: Subject to 12-month waiting period.
Mental health services covered at 80% after deductible.
Emergency room co-payment: $250 per visit (waived if admitted).
Prescription Drug Coverage: Tier 1 $10 / Tier 2 $35 / Tier 3 $75
""",
"life_policy": ""
TERM LIFE INSURANCE POLICY – Policy No. LIF-2024-55921
Insured: John Michael Anderson | Date of Birth: March 15, 1978
Face Amount: $500,000 | Premium: $42.00/month
Term: 20 years (expires 2044) | Beneficiary: Sarah Anderson (spouse)
Grace Period: 31 days from premium due date
Suicide Exclusion: Benefits not payable within first 2 policy years.
Contestability Period: 2 years from policy issue date.
Conversion Option: Convertible to whole life before age 65.
""
}

print('Insurance documents loaded successfully.')
for name, doc in insurance_docs.items():
    print(f' {name}: {len(doc.split())} words')

```

```

Insurance documents loaded successfully.
auto_policy: 60 words
health_policy: 72 words
life_policy: 78 words

```

Step 2 Tokenize with tiktoken (OpenAI tokenizer) — Cell 2

```

# Cell 2 – tiktoken: OpenAI GPT-3.5 and GPT-4 tokenization

import tiktoken

```

```

# Load encodings for different OpenAI models
enc_gpt35 = tiktoken.encoding_for_model('gpt-3.5-turbo') # cl100k_base
enc_gpt4 = tiktoken.encoding_for_model('gpt-4') # cl100k_base
enc_davinci = tiktoken.get_encoding('p50k_base') # text-davinci-003

results = {}

for doc_name, doc_text in insurance_docs.items():
    tokens_gpt35 = enc_gpt35.encode(doc_text)
    tokens_gpt4 = enc_gpt4.encode(doc_text)
    tokens_davinci = enc_davinci.encode(doc_text)
    word_count = len(doc_text.split())

    results[doc_name] = {
        'words': word_count,
        'gpt35': len(tokens_gpt35),
        'gpt4': len(tokens_gpt4),
        'davinci': len(tokens_davinci),
    }

    print(f'\n--- {doc_name} ---')
    print(f' Words : {word_count}')
    print(f' GPT-3.5 tokens : {len(tokens_gpt35)}')
    print(f' GPT-4 tokens : {len(tokens_gpt4)}')
    print(f' Davinci tokens : {len(tokens_davinci)}')
    print(f' Token/Word ratio (GPT-4): {len(tokens_gpt4)/word_count:.2f}')

```

```

--- auto_policy ---
Words : 60
GPT-3.5 tokens : 98
GPT-4 tokens : 98
Davinci tokens : 101
Token/Word ratio (GPT-4): 1.63

--- health_policy ---
Words : 72
GPT-3.5 tokens : 118
GPT-4 tokens : 118
Davinci tokens : 122
Token/Word ratio (GPT-4): 1.64

--- life_policy ---

```

```
Words : 78
GPT-3.5 tokens : 126
GPT-4 tokens : 126
Davinci tokens : 130
Token/Word ratio (GPT-4): 1.62
```

■ *Observation: GPT-3.5 and GPT-4 share the same `cl100k_base` tokenizer, producing identical token counts. The older `p50k_base` (Davinci) produces slightly more tokens because its vocabulary is smaller, so it splits rarer terms into more sub-word pieces.*

Step 3

Tokenize with HuggingFace Tokenizers (BERT and GPT-2) — Cell 3

```
# Cell 3 – HuggingFace Tokenizers: BERT and GPT-2

from transformers import AutoTokenizer

# Load BERT and GPT-2 tokenizers
bert_tokenizer = AutoTokenizer.from_pretrained('bert-base-uncased')
gpt2_tokenizer = AutoTokenizer.from_pretrained('gpt2')

hf_results = {}

for doc_name, doc_text in insurance_docs.items():
    # BERT tokenization (WordPiece algorithm)
    bert_tokens = bert_tokenizer.tokenize(doc_text)
    bert_ids = bert_tokenizer.encode(doc_text)

    # GPT-2 tokenization (BPE algorithm)
    gpt2_tokens = gpt2_tokenizer.tokenize(doc_text)
    gpt2_ids = gpt2_tokenizer.encode(doc_text)

    hf_results[doc_name] = {
        'bert_tokens': len(bert_tokens),
        'gpt2_tokens': len(gpt2_tokens),
    }

    print(f'\n--- {doc_name} ---')
    print(f' BERT tokens : {len(bert_tokens)}')
    print(f' GPT-2 tokens : {len(gpt2_tokens)}')

# Show how BERT handles insurance-specific vocabulary
sample = 'deductible copayment contestability beneficiary'
bert_sample = bert_tokenizer.tokenize(sample)
gpt2_sample = gpt2_tokenizer.tokenize(sample)
```

```
print(f' BERT splits insurance terms: {bert_sample}')
print(f' GPT-2 splits insurance terms: {gpt2_sample}')
```

```
--- auto_policy ---

BERT tokens : 143

GPT-2 tokens : 104

BERT splits insurance terms:
['deduct', '##ible', 'co', '##pay', '##ment', 'contest', '##ability', 'beneficiary']

GPT-2 splits insurance terms:
['deductible', '■copayment', '■contest', 'ability', '■beneficiary']
```

■ *Observation: BERT's WordPiece algorithm adds ## prefixes to sub-word continuations. It produces more tokens than GPT-4 for insurance text because insurance-specific words like 'contestability' are less common in its vocabulary. GPT-2 BPE handles compound insurance terms more efficiently but still produces more tokens than tiktoken.*

Step 4

Visualize token comparison with a bar chart — Cell 4

```
# Cell 4 – Visualization: Token count comparison across models

import matplotlib.pyplot as plt
import numpy as np

doc_names = list(insurance_docs.keys())
doc_labels = ['Auto Policy', 'Health Policy', 'Life Policy']
models = ['GPT-3.5/4', 'Davinci', 'BERT', 'GPT-2']
colors_list = ['#1565C0', '#6A1B9A', '#E65100', '#2E7D32']

# Build token count matrix [4 models x 3 documents]
token_matrix = []
for doc_name in doc_names:
    row = [
        results[doc_name]['gpt4'],
        results[doc_name]['davinci'],
        hf_results[doc_name]['bert_tokens'],
        hf_results[doc_name]['gpt2_tokens'],
    ]
    token_matrix.append(row)

token_matrix = np.array(token_matrix).T # Shape: 4 x 3

x = np.arange(len(doc_labels))
width = 0.18
```

```

fig, ax = plt.subplots(figsize=(11, 6))
fig.patch.set_facecolor('#F0F4F8')
ax.set_facecolor('#F0F4F8')

for i, (model, color) in enumerate(zip(models, colors_list)):
    bars = ax.bar(x + i*width - 1.5*width, token_matrix[i],
width, label=model, color=color, alpha=0.87, edgecolor='white')
    for bar in bars:
        ax.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 1,
str(int(bar.get_height())), ha='center', va='bottom',
fontsize=8, fontweight='bold', color='#333')

ax.set_xlabel('Insurance Document Type', fontsize=11, fontweight='bold')
ax.set_ylabel('Token Count', fontsize=11, fontweight='bold')
ax.set_title('Token Count Comparison: Insurance Documents Across LLM Tokenizers',
fontsize=13, fontweight='bold', pad=15)
ax.set_xticks(x)
ax.set_xticklabels(doc_labels, fontsize=10)
ax.legend(title='Tokenizer / Model', fontsize=9, title_fontsize=9)
ax.grid(axis='y', alpha=0.4, linestyle='--')
ax.spines[['top', 'right']].set_visible(False)

plt.tight_layout()
plt.savefig('token_comparison.png', dpi=150, bbox_inches='tight')
plt.show()
print('Chart saved as token_comparison.png')

```

Expected Chart Output

- Bar chart with 4 grouped bars per document type (3 document types = 12 bars total).
- BERT bars are consistently the tallest (most tokens) due to WordPiece sub-word splitting.
- GPT-3.5 and GPT-4 bars are the shortest (most efficient) using cl100k_base vocabulary.
- GPT-2 sits between BERT and GPT-4 in token efficiency for insurance text.
- Chart is saved as token_comparison.png in the current working directory.

Step 5

Visualize token boundaries with color-coded HTML in Jupyter — Cell 5

```
# Cell 5 – Color-coded token visualization in Jupyter
```



```

from IPython.display import display, HTML
import tiktoken

enc = tiktoken.encoding_for_model('gpt-4')

# Use a focused excerpt from the health policy
sample_clause = (
    'Pre-existing conditions are subject to a 12-month waiting period. '
    'Mental health services are covered at 80 percent after deductible. '
    'Emergency room co-payment is $250 per visit.'
)

token_ids = enc.encode(sample_clause)
token_strings = [enc.decode([t]) for t in token_ids]

# Cycle through distinct background colors
palette = ['#BBDEFB', '#C8E6C9', '#FFE0B2', '#E1BEE7', '#B2EBF2', '#FFCDD2', '#F0F4C3']

html_parts = ['<div
style="font-family:monospace;font-size:14px;line-height:2.2;padding:16px;'
'background:#1E2A3A;border-radius:8px;color:#E8EAF6;">']

for i, token_str in enumerate(token_strings):
    bg = palette[i % len(palette)]
    display_str = token_str.replace(' ', '&nbsp;')
    html_parts.append(
        f'<span style="background:{bg};color:#1A237E;border-radius:3px;'
        f'padding:2px 4px;margin:1px;font-weight:bold" '
        f'title="Token ID: {token_ids[i]}">{{display_str}}</span>'
    )

html_parts.append(f'</div>')

html_parts.append(f'<p style="color:#607D8B;font-size:12px;margin-top:8px;">'
f'Total tokens: {len(token_ids)} &nbsp;|&nbsp;'
f'Words: {len(sample_clause.split())} &nbsp;|&nbsp;'
f'Ratio: {len(token_ids)/len(sample_clause.split()):.2f} tokens/word</p>')

display(HTML(''.join(html_parts)))

```

Expected Output

- A color-coded block where each token has a unique background color.

- Hover over any token to see its numeric Token ID.
- Insurance jargon like 'Pre-existing', 'co-payment' will be visibly split across 2-3 token spans.
- Total token count, word count, and ratio displayed below the visualization.

■ Module 2 — Token Cost Calculation & Prompt Optimization

What You Will Learn

- Calculate the exact API cost for processing insurance documents using current OpenAI pricing.
- Build a reusable cost calculator function for any prompt and model combination.
- Apply concrete prompt optimization techniques that reduce token usage by 30–60%.
- Compare cost at scale: processing 10,000 insurance policies vs optimized prompts.
- Learn system prompt compression without losing LLM performance.

OpenAI Pricing Reference (as of 2024)

Model	Input (per 1M tokens)	Output (per 1M tokens)	Context Window
gpt-4o	\$2.50	\$10.00	128K tokens
gpt-4o-mini	\$0.15	\$0.60	128K tokens
gpt-4-turbo	\$10.00	\$30.00	128K tokens
gpt-3.5-turbo	\$0.50	\$1.50	16K tokens
text-embedding-3-small	\$0.02	N/A	8K tokens
text-embedding-3-large	\$0.13	N/A	8K tokens

■■ Always verify pricing at platform.openai.com/pricing — costs change periodically.

Step 1

Build a reusable token cost calculator — Cell 6

```
# Cell 6 – Token Cost Calculator

import tiktoken

# Current OpenAI pricing (per 1M tokens)
PRICING = {
    'gpt-4o': {'input': 2.50, 'output': 10.00},
    'gpt-4o-mini': {'input': 0.15, 'output': 0.60},
    'gpt-4-turbo': {'input': 10.00, 'output': 30.00},
    'gpt-3.5-turbo': {'input': 0.50, 'output': 1.50},
    'text-embedding-3-small': {'input': 0.02, 'output': 0.00},
    'text-embedding-3-large': {'input': 0.13, 'output': 0.00},
```

```

}

def count_tokens(text: str, model: str = 'gpt-4o') -> int:
    '''Count tokens for a given text and model using tiktoken.'''
    try:
        enc = tiktoken.encoding_for_model(model)
    except KeyError:
        enc = tiktoken.get_encoding('cl100k_base') # fallback
    return len(enc.encode(text))

def calculate_cost(input_text: str, output_text: str = '',
                  model: str = 'gpt-4o') -> dict:
    '''Calculate API cost for a prompt + expected response.'''
    input_tokens = count_tokens(input_text, model)
    output_tokens = count_tokens(output_text, model) if output_text else 0
    pricing = PRICING.get(model, {'input': 0, 'output': 0})

    input_cost = (input_tokens / 1_000_000) * pricing['input']
    output_cost = (output_tokens / 1_000_000) * pricing['output']
    total_cost = input_cost + output_cost

    return {
        'model': model,
        'input_tokens': input_tokens,
        'output_tokens': output_tokens,
        'total_tokens': input_tokens + output_tokens,
        'input_cost_usd': input_cost,
        'output_cost_usd': output_cost,
        'total_cost_usd': total_cost,
    }

print('Cost calculator ready.')

```

Step 2

Calculate cost for an insurance policy analysis task — Cell 7

```

# Cell 7 – Analyze cost of a policy summarization prompt

# Define the system prompt for policy analysis
system_prompt = '''You are an expert insurance policy analyst with 20 years of experience
in auto, health, and life insurance. Your role is to analyze insurance policy documents
and provide clear, structured summaries that help policyholders understand their coverage.
Always identify: coverage amounts, deductibles, exclusions, and key deadlines.

```

```

Format your response as a structured JSON object with these keys:
policy_type, coverage_summary, deductibles, exclusions, key_dates, risk_flags.

# The user prompt includes the actual policy document
user_prompt = f'Please analyze this insurance policy:\n{insurance_docs["health_policy"]}'

# Expected output (approximate – what GPT-4 would return)
expected_output = '''{
  "policy_type": "Health Insurance - PPO",
  "coverage_summary": {"type": "PPO", "premium": "$425/month"},
  "deductibles": {"individual": "$1,500", "oop_max": "$5,000"},
  "exclusions": ["Pre-existing conditions (12-month wait)"],
  "key_dates": "Calendar year reset",
  "risk_flags": ["High deductible", "Pre-existing condition waiting period"]
}'''

full_input = system_prompt + '\n' + user_prompt

# Calculate cost across different models
for model in ['gpt-4o', 'gpt-4o-mini', 'gpt-3.5-turbo']:
    cost = calculate_cost(full_input, expected_output, model)
    print(f'\nModel: {cost["model"]}')
    print(f' Input tokens : {cost["input_tokens"]}')
    print(f' Output tokens : {cost["output_tokens"]}')
    print(f' Total cost : ${cost["total_cost_usd"]:.6f}')
    print(f' Cost per 1000 docs: ${cost["total_cost_usd"]*1000:.3f}')

```

```

Model: gpt-4o
Input tokens : 312
Output tokens : 118
Total cost : $0.001960
Cost per 1000 docs: $1.960

Model: gpt-4o-mini
Input tokens : 312
Output tokens : 118
Total cost : $0.000117
Cost per 1000 docs: $0.117

Model: gpt-3.5-turbo
Input tokens : 312
Output tokens : 118

```

Total cost : \$0.000333
Cost per 1000 docs: \$0.333

Step 3 Apply prompt optimization techniques — Cell 8

The original system prompt uses 107 tokens and is verbose. We can reduce this significantly using three optimization strategies without losing output quality.

	Description	Token Savings
es	Delete 'Your role is to', 'Always', flowery descriptions	~15-25%
s	Replace prose instructions with concise imperative bullets	~20-30%
only	Ask for JSON keys without explaining what each key means	~10-15%
cy	Don't repeat the task framing already in the user prompt	~5-10%

```
# Cell 8 – Prompt optimization: before and after comparison

# ORIGINAL system prompt (107 tokens)
original_system = '''You are an expert insurance policy analyst with 20 years of experience
in auto, health, and life insurance. Your role is to analyze insurance policy documents
and provide clear, structured summaries that help policyholders understand their coverage.
Always identify: coverage amounts, deductibles, exclusions, and key deadlines.
Format your response as a structured JSON object with these keys:
policy_type, coverage_summary, deductibles, exclusions, key_dates, risk_flags.'''

# OPTIMIZED system prompt (same meaning, fewer tokens)
optimized_system = '''Insurance policy analyst. Respond in JSON:
{policy_type, coverage_summary, deductibles, exclusions, key_dates, risk_flags}'''

# OPTIMIZED user prompt: remove boilerplate, be direct
original_user = f'Please analyze this insurance policy:\n{insurance_docs["health_policy"]}'
optimized_user = f'Analyze:\n{insurance_docs["health_policy"].strip()}'

# Count tokens for both versions
orig_tokens = count_tokens(original_system + original_user)
opt_tokens = count_tokens(optimized_system + optimized_user)
savings_pct = (orig_tokens - opt_tokens) / orig_tokens * 100

print(f'Original prompt : {orig_tokens} tokens')
print(f'Optimized prompt : {opt_tokens} tokens')
print(f'Token savings : {orig_tokens - opt_tokens} tokens ({savings_pct:.1f}%)')

# Project savings at enterprise scale
```

```

for scale in [1_000, 10_000, 100_000]:
    orig_cost = calculate_cost(original_system + original_user, '', 'gpt-4o')['input_cost_usd']
    opt_cost = calculate_cost(optimized_system + optimized_user, '', 'gpt-4o')['input_cost_usd']
    saved = (orig_cost - opt_cost) * scale
    print(f' {scale:>7,} docs → save ${saved:.2f} on gpt-4o input alone')

```

```

Original prompt : 312 tokens
Optimized prompt : 196 tokens
Token savings : 116 tokens (37.2%)

1,000 docs → save $0.29 on gpt-4o input alone
10,000 docs → save $2.90 on gpt-4o input alone
100,000 docs → save $29.00 on gpt-4o input alone

```

Step 4 Visualize cost at scale — Cell 9

```

# Cell 9 – Cost at scale visualization

import matplotlib.pyplot as plt
import numpy as np

scales = [1_000, 5_000, 10_000, 50_000, 100_000]
models = ['gpt-4o', 'gpt-4o-mini', 'gpt-3.5-turbo']
colors = ['#1565C0', '#2E7D32', '#E65100']

fig, axes = plt.subplots(1, 2, figsize=(13, 5))
fig.patch.set_facecolor('#F0F4F8')

for ax, prompt_type, title in zip(
    axes,
    [(original_system + original_user, 'Original'),
     (optimized_system + optimized_user, 'Optimized')],
    ['Original Prompt Cost at Scale', 'Optimized Prompt Cost at Scale']
):
    ax.set_facecolor('#F0F4F8')
    for model, color in zip(models, colors):
        costs = []
        for scale in scales:
            c = calculate_cost(prompt_type[0], expected_output, model)
            costs.append(c['total_cost_usd'] * scale)
        ax.plot([s/1000 for s in scales], costs,
            marker='o', label=model, color=color, linewidth=2)

```

```
ax.set_xlabel('Documents Processed (thousands)', fontsize=10)
ax.set_ylabel('Total API Cost (USD)', fontsize=10)
ax.set_title(title, fontsize=11, fontweight='bold')
ax.legend(fontsize=9)
ax.grid(alpha=0.3, linestyle='--')
ax.spines[['top', 'right']].set_visible(False)

plt.suptitle('Insurance Policy Analysis – API Cost vs Scale', fontsize=13, fontweight='bold')
plt.tight_layout()
plt.savefig('cost_at_scale.png', dpi=150, bbox_inches='tight')
plt.show()
```

Expected Chart

- Two side-by-side line charts: Original vs Optimized prompt costs at scale.
- X-axis: 1K, 5K, 10K, 50K, 100K documents. Y-axis: total cost in USD.
- gpt-4o rises steeply on both charts; gpt-4o-mini stays very low.
- The Optimized chart shows ~37% lower costs across all models and scales.

■ Module 3 — Embeddings & Semantic Similarity for Policy Clauses

What You Will Learn

- Understand what embeddings are and how they represent text as high-dimensional vectors.
- Generate embeddings for insurance policy clauses using both OpenAI and SentenceTransformers.
- Compute cosine similarity to find semantically similar policy clauses.
- Build a clause search engine that finds the most relevant policy section for a user query.
- Visualize embedding clusters using PCA and a heatmap of clause similarities.

Concept: What Are Embeddings?

An embedding is a fixed-length numeric vector (e.g., 1,536 numbers for OpenAI ada-002) that encodes the semantic meaning of text. Two pieces of text with similar meaning will produce vectors that are close together in vector space — measurable using cosine similarity (1.0 = identical meaning, 0.0 = unrelated, -1.0 = opposite). This enables semantic search, clause deduplication, and policy comparison at enterprise scale.

Model	Dimensions	Max Input	Cost per 1M tokens	Best For
text-embedding-3-small	1,536	8,192 tokens	\$0.02	High-volume, c
text-embedding-3-large	3,072	8,192 tokens	\$0.13	High-accuracy
all-MiniLM-L6-v2 (HF)	384	512 tokens	Free (local)	Fast, offline us
all-mpnet-base-v2 (HF)	768	384 tokens	Free (local)	Accuracy, offli

Step-by-Step Implementation

Step 1 Extract policy clauses from insurance documents — Cell 10

```
# Cell 10 – Define insurance policy clauses for embedding

# Individual clauses extracted from policy documents
policy_clauses = {
    'auto_deductible': 'The insured must pay a $500 deductible per collision claim before
insurance coverage applies.',
    'auto_exclusions': 'This policy excludes coverage for racing, commercial use, and intentional
vehicle damage.'
```

```

'auto_claims_window': 'All accident claims must be reported to the insurer within 30 days of
the incident.',

'health_deductible': 'The annual health insurance deductible is $1,500 per individual insured
member.',

'health_oop_max': 'The maximum out-of-pocket expense per calendar year is $5,000 per insured
individual.',

'health_preexisting': 'Pre-existing medical conditions are subject to a mandatory 12-month
waiting period.',

'health_mental': 'Mental health and behavioral health services are covered at 80 percent after
the annual deductible.',

'health_er': 'Emergency room visits require a $250 copayment, waived when the patient is
subsequently admitted.',

'life_face_amount': 'The death benefit payable to the named beneficiary is $500,000 upon death
of the insured.',

'life_grace_period': 'A 31-day grace period is granted for premium payments after the due
date.',

'life_suicide': 'Death by suicide is excluded from coverage within the first two years of
policy issuance.',

'life_contestability': 'The insurer may contest or void this policy within the first two years
from the policy issue date.',

}

clauses_list = list(policy_clauses.values())
clause_names = list(policy_clauses.keys())

print(f'Total policy clauses: {len(clauses_list)}')

for name, clause in policy_clauses.items():
    print(f' [{name:25s}] {clause[:60]}...')

```

```

Total policy clauses: 12

[auto_deductible ] The insured must pay a $500 deductible per collision...
[auto_exclusions ] This policy excludes coverage for racing, commercial...
[auto_claims_window ] All accident claims must be reported to the insurer...
[health_deductible ] The annual health insurance deductible is $1,500 per...
[health_oop_max ] The maximum out-of-pocket expense per calendar year...
... (12 clauses total)

```

Step 2

Generate embeddings using SentenceTransformers (free, local) — Cell 11

```

# Cell 11 – SentenceTransformers embeddings (runs locally, no API key needed)

from sentence_transformers import SentenceTransformer

import numpy as np

from sklearn.metrics.pairwise import cosine_similarity

```

```

# Load the model (downloads ~90MB on first run, cached afterwards)
print('Loading SentenceTransformer model...')
st_model = SentenceTransformer('all-MiniLM-L6-v2')
print('Model loaded successfully.')

# Generate embeddings for all 12 policy clauses
print('Generating embeddings for policy clauses...')
st_embeddings = st_model.encode(clauses_list, show_progress_bar=True)

print(f'\nEmbedding matrix shape: {st_embeddings.shape}')
print(f'Each clause → vector of {st_embeddings.shape[1]} dimensions')

# Compute full cosine similarity matrix (12 x 12)
sim_matrix = cosine_similarity(st_embeddings)

print('\nTop 5 most similar clause pairs (SentenceTransformers):')
pairs = []
for i in range(len(clause_names)):
    for j in range(i+1, len(clause_names)):
        pairs.append((sim_matrix[i,j], clause_names[i], clause_names[j]))

pairs.sort(reverse=True)
for score, c1, c2 in pairs[:5]:
    print(f' {score:.4f} | {c1} ↔ {c2}')

```

```

Loading SentenceTransformer model...
Model loaded successfully.
Generating embeddings for policy clauses...
Embedding matrix shape: (12, 384)
Each clause → vector of 384 dimensions

Top 5 most similar clause pairs (SentenceTransformers):
0.8342 | auto_deductible ↔ health_deductible
0.7891 | health_oop_max ↔ health_deductible
0.7210 | life_suicide ↔ life_contestability
0.6954 | auto_claims_window ↔ life_grace_period
0.6721 | health_er ↔ auto_deductible

```

■ *Observation: auto_deductible and health_deductible score 0.83 similarity — both describe cost-sharing mechanisms. Life policy exclusions cluster together. Cross-policy financial terms (deductibles, copayments, limits) show high similarity regardless of insurance type.*

```

# Cell 12 – OpenAI Embeddings API

import openai
import os

client = openai.OpenAI(api_key=os.environ.get('OPENAI_API_KEY'))

def get_openai_embeddings(texts: list, model: str = 'text-embedding-3-small') -> np.ndarray:
    '''Generate OpenAI embeddings for a list of texts.'''
    response = client.embeddings.create(input=texts, model=model)
    embeddings = [item.embedding for item in response.data]
    return np.array(embeddings)

print('Generating OpenAI embeddings...')
oai_embeddings = get_openai_embeddings(clauses_list, 'text-embedding-3-small')

print(f'OpenAI embedding shape: {oai_embeddings.shape}')
print(f'Each clause → vector of {oai_embeddings.shape[1]} dimensions')

# Compute cost of this embedding operation
total_text = ' '.join(clauses_list)
emb_cost = calculate_cost(total_text, '', 'text-embedding-3-small')
print(f'Cost to embed 12 clauses: ${emb_cost["input_cost_usd"]:.6f}')
print(f'Cost to embed 10,000 docs (12 clauses each):
${emb_cost["input_cost_usd"]*10000:.4f}')

# Compute similarity matrix
oai_sim_matrix = cosine_similarity(oai_embeddings)

print('\nTop 5 most similar clause pairs (OpenAI text-embedding-3-small):')
pairs_oai = []
for i in range(len(clause_names)):
    for j in range(i+1, len(clause_names)):
        pairs_oai.append((oai_sim_matrix[i,j], clause_names[i], clause_names[j]))
pairs_oai.sort(reverse=True)
for score, c1, c2 in pairs_oai[:5]:
    print(f' {score:.4f} | {c1} ↔ {c2}')

```

```

Generating OpenAI embeddings...
OpenAI embedding shape: (12, 1536)
Each clause → vector of 1536 dimensions
Cost to embed 12 clauses: $0.000004
Cost to embed 10,000 docs (12 clauses each): $0.0415

```

Top 5 most similar clause pairs (OpenAI text-embedding-3-small):

```
0.8871 | auto_deductible ↔ health_deductible
0.8203 | health_oop_max ↔ health_deductible
0.7654 | life_suicide ↔ life_contestability
0.7340 | auto_exclusions ↔ life_suicide
0.7198 | health_er ↔ health_oop_max
```

■ OpenAI's 1,536-dimensional embedding is more discriminating — the top similarity scores are higher (0.88 vs 0.83) showing tighter semantic clustering for related clauses. Embedding 10,000 complete policies with 12 clauses each costs only ~\$0.04.

Step 4

Build a clause semantic search engine — Cell 13

```
# Cell 13 — Semantic search over policy clauses

def semantic_search(query: str, embeddings: np.ndarray,
                    clause_names: list, clauses_list: list,
                    top_k: int = 3, use_openai: bool = False) -> list:
    '''Find the most semantically similar clauses to a query.'''
    if use_openai:
        query_emb = get_openai_embeddings([query])[0]
    else:
        query_emb = st_model.encode([query])[0]

    # Compute cosine similarity between query and all clauses
    sims = cosine_similarity([query_emb], embeddings)[0]

    # Get top-k results
    top_indices = sims.argsort()[::-1][:top_k]
    results = []
    for idx in top_indices:
        results.append({
            'rank': len(results) + 1,
            'clause_name': clause_names[idx],
            'clause_text': clauses_list[idx],
            'similarity': float(sims[idx]),
        })
    return results

# --- Test queries simulating real policyholder questions ---
test_queries = [
    'How much do I need to pay before my coverage starts?',
```

```

'What happens if I miss a premium payment?',
'Are mental health treatments covered under my plan?',
'Can my insurance company cancel my policy?',
]

for query in test_queries:
    print(f'\n■ Query: "{query}"')
    results = semantic_search(query, st_embeddings, clause_names, clauses_list, top_k=2)
    for r in results:
        print(f' # {r["rank"]} [{r["similarity"]:.4f}] {r["clause_name"]}')
        print(f' {r["clause_text"][:90]}...')

```

```

■ Query: "How much do I need to pay before my coverage starts?"
#1 [0.7842] auto_deductible
The insured must pay a $500 deductible per collision claim before insurance...
#2 [0.7701] health_deductible
The annual health insurance deductible is $1,500 per individual insured...

■ Query: "What happens if I miss a premium payment?"
#1 [0.8123] life_grace_period
A 31-day grace period is granted for premium payments after the due date.
#2 [0.6234] auto_claims_window
All accident claims must be reported to the insurer within 30 days...

■ Query: "Are mental health treatments covered under my plan?"
#1 [0.8956] health_mental
Mental health and behavioral health services are covered at 80 percent...
#2 [0.6102] health_er
Emergency room visits require a $250 copayment, waived when the patient...

■ Query: "Can my insurance company cancel my policy?"
#1 [0.7432] life_contestability
The insurer may contest or void this policy within the first two years...
#2 [0.6891] life_suicide
Death by suicide is excluded from coverage within the first two policy years.

```

Step 5

Visualize similarity heatmap and PCA cluster — Cell 14

```

# Cell 14 – Heatmap and PCA visualization

import matplotlib.pyplot as plt

import seaborn as sns

```

```

from sklearn.decomposition import PCA
import numpy as np

fig, axes = plt.subplots(1, 2, figsize=(15, 6))
fig.patch.set_facecolor('#F0F4F8')

# --- Plot 1: Cosine Similarity Heatmap ---
ax1 = axes[0]
ax1.set_facecolor('#F0F4F8')
short_names = [n.replace('_', '\n') for n in clause_names]
sns.heatmap(
    sim_matrix, ax=ax1,
    xticklabels=short_names, yticklabels=short_names,
    cmap='Blues', annot=True, fmt='.2f', annot_kws={'size': 7},
    linewidths=0.5, linecolor='#CFD8DC', vmin=0, vmax=1,
    cbar_kws={'label': 'Cosine Similarity'})
)
ax1.set_title('Policy Clause Similarity Heatmap\n(SentenceTransformers)', fontsize=11,
fontweight='bold')
ax1.tick_params(axis='both', labelsize=7)

# --- Plot 2: PCA 2D Cluster ---
ax2 = axes[1]
ax2.set_facecolor('#F0F4F8')
pca = PCA(n_components=2)
coords = pca.fit_transform(st_embeddings)

policy_types = ['auto']*3 + ['health']*5 + ['life']*4
color_map = {'auto': '#1565C0', 'health': '#2E7D32', 'life': '#E65100'}
marker_map = {'auto': 'o', 'health': 's', 'life': '^'}

for ptype in ['auto', 'health', 'life']:
    mask = [t == ptype for t in policy_types]
    xs = coords[mask, 0]
    ys = coords[mask, 1]
    ax2.scatter(xs, ys, c=color_map[ptype], marker=marker_map[ptype],
s=120, label=f'{ptype.capitalize()} Policy', alpha=0.85,
edgecolors='white', linewidths=1.5)

# Annotate each point with clause name
for i, name in enumerate(clause_names):

```

```

short = name.split('_', 1)[1] # Remove policy type prefix
ax2.annotate(short, (coords[i, 0], coords[i, 1]),
textcoords='offset points', xytext=(5, 5), fontsize=7.5)

ax2.set_title(f'Policy Clauses in 2D Embedding Space\n(PCA of 384-dim vectors)',
fontsize=11, fontweight='bold')
ax2.legend(fontsize=9)
ax2.set_xlabel(f'PCA Component 1 ({pca.explained_variance_ratio_[0]*100:.1f}% variance)')
ax2.set_ylabel(f'PCA Component 2 ({pca.explained_variance_ratio_[1]*100:.1f}% variance)')
ax2.grid(alpha=0.3, linestyle='--')
ax2.spines[['top', 'right']].set_visible(False)

plt.tight_layout()
plt.savefig('embeddings_visualization.png', dpi=150, bbox_inches='tight')
plt.show()
print('Visualization saved as embeddings_visualization.png')

```

Expected Visualization

- LEFT — Heatmap: 12×12 grid with cosine similarity scores. Bright blue = high similarity.
- Auto/health deductible blocks will glow bright. Cross-type exclusions will show medium similarity.
- RIGHT — PCA scatter: Three distinct clusters visible — auto (circles), health (squares), life (triangles).
- Health OOP Max and Health Deductible will appear close. Life Suicide and Contestability cluster together.
- The PCA axes show how much variance is captured by the 2D projection (typically 35–55%).

■ Capstone Exercise — End-to-End Policy Intelligence Pipeline

Combine all three modules into one reusable pipeline: tokenize → cost-check → embed → search.

```
# Cell 15 – Capstone: Full Insurance Policy Intelligence Pipeline
```

```
def analyze_policy(policy_text: str, user_query: str,
model: str = 'gpt-4o-mini') -> dict:
    '''
```

```
    End-to-end pipeline:
```

1. Count tokens and estimate API cost
2. Warn if document exceeds budget threshold
3. Generate embedding for the document
4. Find most similar existing policy clauses
5. Return analysis report

```
    '''
```

```
# Step 1: Token count and cost
```

```
token_count = count_tokens(policy_text, model)
cost_info = calculate_cost(policy_text, '', model)
```

```
# Step 2: Cost gate – warn if single doc costs > $0.01
```

```
budget_ok = cost_info['total_cost_usd'] < 0.01
```

```
# Step 3: Embed the policy document
```

```
doc_embedding = st_model.encode([policy_text])
```

```
# Step 4: Find similar clauses
```

```
similar = semantic_search(
    user_query, st_embeddings, clause_names, clauses_list, top_k=3
)
```

```
return {
    'token_count': token_count,
    'estimated_cost_usd': cost_info['total_cost_usd'],
    'within_budget': budget_ok,
    'model': model,
    'similar_clauses': similar,
}
```

```
# Run the pipeline on a new policy document
```

```

new_policy = insurance_docs['auto_policy']

query = 'What are my payment obligations after an accident?'

print('Running Insurance Policy Intelligence Pipeline...')

result = analyze_policy(new_policy, query, model='gpt-4o-mini')

print(f'\n=== Pipeline Report ===')

print(f'Model : {result["model"]}')

print(f'Token Count : {result["token_count"]}')

print(f'Est. Cost : ${result["estimated_cost_usd"]:.6f}')

print(f'Within Budget : {result["within_budget"]}')

print(f'\nQuery: "{query}"')

print(f'\nTop Matching Clauses:')

for c in result['similar_clauses']:

    print(f' [{c["similarity"]:.4f}] {c["clause_name"]}: {c["clause_text"][:80]}...')

```

```

Running Insurance Policy Intelligence Pipeline...

=== Pipeline Report ===

Model : gpt-4o-mini

Token Count : 98

Est. Cost : $0.000015

Within Budget : True

Query: "What are my payment obligations after an accident?"

Top Matching Clauses:

[0.7912] auto_deductible: The insured must pay a $500 deductible per collision...

[0.7234] auto_claims_window: All accident claims must be reported within 30 days...

[0.6891] health_er: Emergency room visits require a $250 copayment...

```

Capstone Task	Implementation Hint
Add a new commercial property policy document	Extend insurance_docs dict with 3+ new clauses
Compare gpt-4o vs gpt-4o-mini output quality	Call client.chat.completions.create() with both
Cache embeddings to disk to avoid recomputation	Use np.save() / np.load() with a metadata JSON
Build a Gradio UI for the search engine	pip install gradio — wrap semantic_search() in gr.Interface
Add FAISS for fast large-scale similarity search	pip install faiss-cpu — index.add(embeddings)

■ ■ Troubleshooting Common Errors

■ **AuthenticationError: No API key provided**

■ Fix: Ensure OPENAI_API_KEY is set. In Jupyter: `import os; os.environ['OPENAI_API_KEY']='sk-...'`

■ **ModuleNotFoundError: No module named 'tiktoken'**

■ Fix: Run: `pip install tiktoken transformers sentence-transformers openai` in a terminal cell (prefix with !)

■ **RateLimitError from OpenAI API**

■ Fix: Add `time.sleep(1)` between API calls, or use batch mode. For embeddings, batch all texts in one call.

■ **BERT tokenizer: Token indices sequence length > 512**

■ Fix: BERT max is 512 tokens. Truncate: `bert_tokenizer.encode(text, max_length=512, truncation=True)`

■ **PCA visualization looks flat (one cluster)**

■ Fix: Normalize embeddings first: `from sklearn.preprocessing import normalize; embeddings = normalize(embeddings)`

■ **Memory error during SentenceTransformer encoding**

■ Fix: Encode in batches: `st_model.encode(clauses_list, batch_size=16, show_progress_bar=True)`

■ **Port 8888 not accessible in browser**

■ Fix: Check EC2 Security Group: add inbound rule TCP 8888 from 0.0.0.0/0 (or your IP for security)

■ **Lab Summary — Key Takeaways**

Module	Key Skill	Primary Tool	Business Impact
1 — Tokenization	Visualize how insurance text is split into tokens and how many tokens	<code>tiktoken</code> , <code>HuggingFace</code>	Understand why insurance jargon cost
2 — Cost Calc	Calculate and optimize API cost for policy analysis prompts at scale	<code>tokens</code> , <code>promptable</code>	37% token reduction = \$29 saved per 1000 prompts
3 — Embeddings	Generate semantic vectors, compute similarity between clauses, OpenAI	<code>SentenceTransformers</code> , <code>OpenAI</code>	Find matching clauses with 0.88 similarity
Capstone	End-to-end pipeline: tokenize → cost-gate → embed → semantic search	All tools combined	Production-ready foundation for insurance search